Python in the Physical Chemistry Lab

[pyPhysChem]

🇫🇷 Utiliser et modifier un notebook sur Google Colab

Mode d'emploi = comment exécuter un notebook dans le cloud, sans rien installer sur votre PC local.

On prend ici l'exemple de « Python in the Physical Chemistry Lab », alias PPCL.ipynb. La plupart des instructions et commentaires sont transférables à tout notebook.

Google Colab permet d'exécuter ce notebook directement dans votre navigateur, sur les serveurs de Google, sans installer Python ni aucune bibliothèque sur votre ordinateur. C'est une alternative pratique à une installation locale de JupyterLab, en particulier sur une machine où vous ne pouvez rien installer. Il vous suffit d'un compte Google.

> **Un notebook Python peut être vu, en première approche, comme une super calculatrice graphique : on y tape des expressions, on obtient des résultats immédiats, et on peut enchaîner et réutiliser les calculs à volonté.**

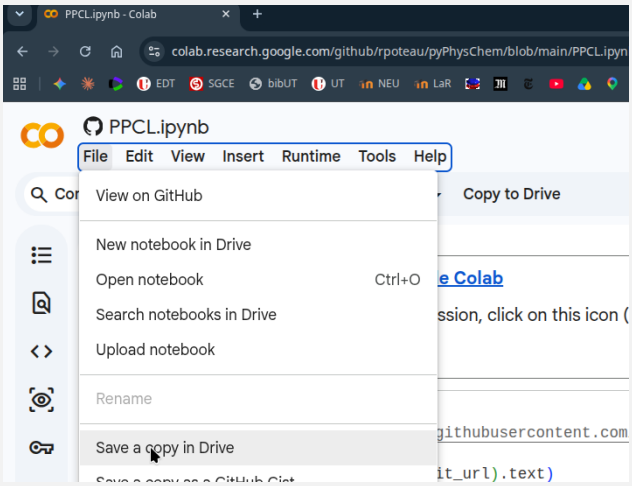
> ***Objectif du notebook PPCL… comme de tout autre notebook servant de support numérique à des cours/TD.*** Il ne s'agit **pas** de devenir un pro de l'algorithmique et de Python. L'objectif est de savoir **écrire du code de base**, et surtout de savoir **lire, interpréter, analyser et corriger** un code fourni par ailleurs, par exemple par une **IA générative**. À l'heure où l'on obtient du code en quelques secondes, l'essentiel n'est plus de le produire, mais de **comprendre ce qu'il fait et de juger s'il est correct**. La **plus-value du support PPCL**, par rapport à d'autres méthodes d'apprentissage de Python, est justement que tout se fait **dans un contexte chimique** : chaque notion est le plus souvent mise en pratique sur de vrais problèmes de chimie physique.
>
> Enfin, les notebooks mis à votre disposition sont **souvent rédigés en anglais** : c'est une excellente occasion, pour des francophones, de se familiariser avec **l'anglais scientifique**, langue internationale incontournable dans un contexte recherche/développement/innovation.

1. La première fois : ouvrir et s'approprier le notebook

1. Rendez-vous sur la page du notebook, par exemple : https://github.com/rpoteau/pyPhysChem/blob/main/PPCL.ipynb (remplacer par le lien de votre notebook de travail)
2. Cliquez sur le badge **« Open in Colab »** en haut du notebook. Connectez-vous avec votre compte Google si ce n'est pas déjà fait.



3. Le notebook s'ouvre dans Colab, mais en **lecture seule** (il vient de GitHub). Pour pouvoir sauvegarder votre travail, allez dans `File > Save a copy in Drive`.



4. Désormais, vous travaillez sur **cette copie**, enregistrée dans votre espace **Google Drive**. C'est elle qu'il faudra rouvrir à chaque séance.

> **À retenir :** cette étape ne se fait qu'**une seule fois**. Ensuite, votre copie personnelle vit dans votre Drive, indépendante de la version GitHub.

2. Reprendre le travail lors d'une séance suivante

1. Allez directement sur colab.research.google.com et connectez-vous.
2. Votre **copie de PPCL** apparaît **en tête des fichiers récents**, sur la page d'accueil. Cliquez dessus pour la rouvrir.



> **Important :** à chaque réouverture, vous devez **ré-exécuter les deux premières cellules d'initialisation** (voir §5). Colab efface l'environnement entre deux sessions.

3. Comprendre le document

Ce document est un **notebook** : il mêle dans une même page du **texte**, des **images**, des **équations** et du **code** que vous pouvez exécuter. Il est composé de **cellules**, de deux types :

- **Cellules de code :** pour écrire et exécuter du Python. Le résultat s'affiche juste en dessous.
- **Cellules de texte (Markdown) :** pour les explications, titres, figures et équations. Si vous double-cliquez dessus par erreur, vous verrez le code de mise en forme : ré-exécutez la cellule pour retrouver l'affichage normal.

4. Exécuter les cellules

Dans Colab, **rien ne s'exécute automatiquement** : écrire $a = 2$ dans une cellule ne crée pas la variable pour autant : tant que vous n'avez pas exécuté la cellule, a n'existe pas. C'est **à vous de lancer chaque cellule**.

- Cliquez dans une cellule, puis appuyez sur **Maj + Entrée** (`Shift + Enter`) : la cellule s'exécute et le curseur passe à la suivante.
- Autre raccourci : **Ctrl + Entrée** exécute la cellule **sans** passer à la suivante.
- Vous pouvez aussi cliquer sur le **bouton ▶️ (play)** à gauche de la cellule.



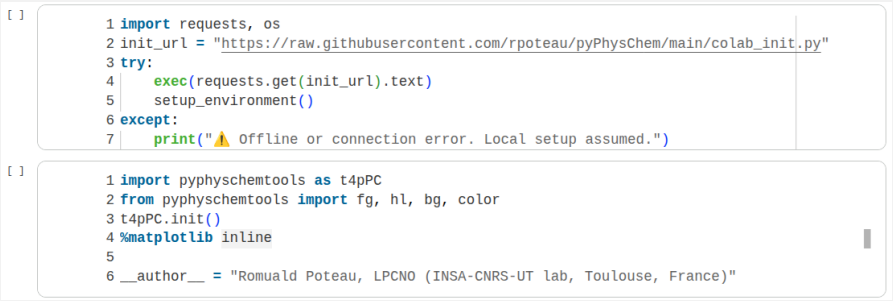
- Pendant l'exécution, le bouton play s'anime. Quand c'est terminé, les crochets à gauche `[ ]` deviennent un `[n]` (un **numéro**) indiquant l'ordre dans lequel les cellules ont été exécutées ; un `[ ]` vide signifie qu'elle n'a pas encore été exécutée.
- Vous pouvez **continuer à modifier d'autres cellules** pendant qu'une cellule longue tourne. En revanche, **une seule cellule s'exécute à la fois** : si vous en lancez d'autres, elles sont mises **en file d'attente** et s'exécuteront l'une après l'autre.
- Pour tout relancer proprement depuis le début : menu `Runtime > Run all`.

> **Bon réflexe :** l'ordre d'exécution compte. Un notebook garde en mémoire ce que les cellules **déjà exécutées** ont produit. Si un résultat semble incohérent, réexécutez depuis le haut (ou `Runtime > Restart session and run all`).

5. Initialiser l'environnement (à faire à chaque session)

> **Le noyau (kernel).** Les cellules de code sont exécutées par un **noyau Python** qui tourne en arrière-plan et **garde en mémoire toutes les variables** que vous avez créées. C'est pour cela qu'une variable définie dans une cellule reste disponible dans les suivantes—à condition de les avoir exécutées. Redémarrer le noyau (`Runtime > Restart session`) vide cette mémoire : il faut alors tout ré-exécuter depuis le début.

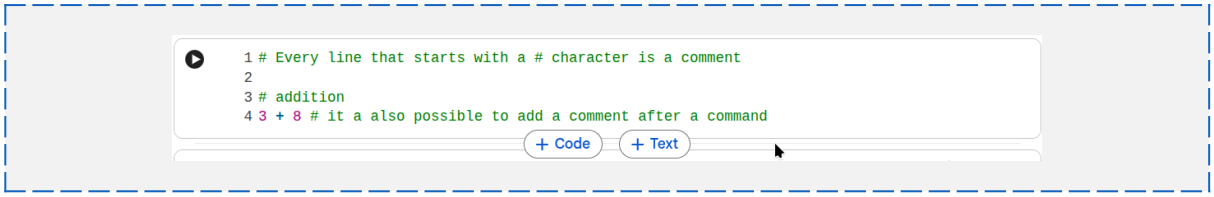
Une fois le notebook ouvert, vous devez préparer l'environnement Python et pyPhysChem en exécutant **les deux premières cellules de code, dans l'ordre**. La première **installe les bibliothèques et télécharge les fichiers d'accompagnement** : elle prend **environ 2 minutes**. Attendez qu'elle soit terminée avant de continuer.



> **Pourquoi refaire cela à chaque fois que le notebook est rouvert ?** Colab tourne sur une **machine virtuelle temporaire** : après une période d'inactivité ou à la fermeture, elle est réinitialisée. Les bibliothèques installées et les fichiers téléchargés sont alors perdus ; mais votre notebook (code et résultats) reste sauvegardé dans votre Drive.

6. Insérer une nouvelle cellule (texte ou code)

Vous pouvez ajouter vos propres cellules, par exemple pour prendre des notes ou tester du code. Survolez l'espace entre deux cellules (ou le bas d'une cellule) : deux boutons apparaissent, **« + Code »** et **« + Texte »**. Cliquez sur celui voulu pour insérer une cellule à cet endroit.



- **+ Code** insère une cellule de code, exécutable avec `Maj + Entrée`.
- **+ Texte** insère une cellule de texte (Markdown) : tapez votre texte, puis `Maj + Entrée` pour l'afficher mis en forme.

7. Applications et exercices

Certains notebooks contiennent des exercices et applications, avec parfois des corrections intégrées. C'est le cas de PPCL.ipynb.

Les applications

Le notebook PPCL contient des **applications** : des exemples de code parfois **assez élaborés**. On ne vous demande **pas** de savoir les écrire vous-même. Leur objectif est de vous faire **comprendre ce que fait le code**, d'en saisir la logique et les différentes parties, et surtout de **prendre du recul** pour être capable de **transposer ces idées à d'autres problèmes**.

Les exercices

Le notebook peuvent aussi contenir des **exercices** sans solution proposée**.** Ils sont alors suivis d'une **cellule de code vide** dans laquelle écrire votre réponse. Ceux-ci sont à faire **par vous-même** : autant que possible, **évitez de recourir à une IA générative** — c'est en cherchant que vous acquerrez les réflexes de base.

La **solution est systématiquement fournie juste après**, mais masquée. La cellule de solution contient une ligne du type `# %load ./SolutionsToExercises/...`. Pour la révéler :

- enlevez le commentaire (# suivi de l'espace) de la ligne `%load` (raccourci clavier utile : `Ctrl + /`) ;
- **exécutez la cellule une première fois** (`Maj + Entrée`) : cela charge le corrigé dans la cellule et remet le commentaire au début de la ligne `%load` …;
- **exécutez-la une seconde fois** : cela fait tourner le code et affiche le résultat.

> **Le bon usage :** essayez d'abord **sérieusement**, puis comparez avec la solution. Révéler la réponse trop tôt vous prive de l'essentiel — apprendre à écrire et à corriger le code vous-même.

8. Sauvegarder votre travail

Une fois votre copie créée dans le Drive (§1), enregistrez à tout moment avec **Ctrl + S** ou `File > Save`. Votre code et les résultats affichés sont conservés. En revanche, le **contenu de la mémoire** (les variables) ne se sauvegarde pas : à la réouverture, réexécutez les cellules depuis le début.

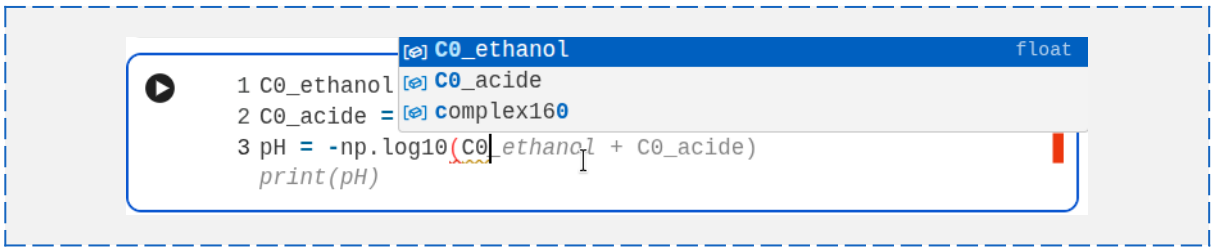
9. Erreurs fréquentes et bonnes habitudes

Python est **exigeant sur la forme** du code. Quelques erreurs reviennent souvent chez les débutants ; les connaître fait gagner beaucoup de temps.

- **L'indentation a un sens.** En Python, les espaces en début de ligne ne sont pas décoratifs : ils délimitent les blocs (à l'intérieur d'un `if`, d'une `for`, d'une fonction…). Une indentation incohérente

déclenche une `IndentationError`. Utilisez toujours le même pas (4 espaces) et ne mélangez pas espaces et tabulations.

- **Pas d'espace superflu.** Évitez un espace en trop en début de ligne, ou avant / après un opérateur de façon anarchique. Un espace involontaire devant une instruction (`a = 2`) peut suffire à provoquer une erreur.
- **Un code bien organisé.** Aérez : une instruction par ligne, des noms de variables parlants, et des commentaires (#) pour expliquer les étapes. Un code lisible est un code que vous (et une IA) pourrez relire et corriger facilement.
- **N'hésitez pas à donner des noms explicites, même longs.** Un nom comme `C0_ethanol` est bien plus clair que `c`. Pas d'inquiétude pour la frappe lorsque vous voudrez réutiliser la variable : comme la plupart des éditeurs, Google Colab propose l'**auto-complétion**. Commencez à taper le début d'un nom, puis appuyez sur `Tab` : Colab complète le nom, ou propose une liste de suggestions si plusieurs noms commencent pareil (naviguer avec les flèches ou bien avec la souris, puis valider avec `Entrée`).



- **Respecter la casse.** Python distingue majuscules et minuscules : `Temperature` et `temperature` sont deux variables différentes.
- **Exécuter dans l'ordre.** Une variable ou une bibliothèque utilisée avant d'avoir été définie / importée donne une `NameError`. Réexécutez depuis le haut en cas de doute.

Exemple : la même ligne, mal puis bien indentée.

```
  x = 2       # espace superflu en début de ligne  →  IndentationError
x = 2         # correct
```

> **Lire le message d'erreur.** Quand une cellule échoue, Python affiche un **message rouge** (un « traceback »). Lisez-le **de bas en haut** : la dernière ligne indique le type d'erreur (`IndentationError`, `NameError`…) et un court résumé. C'est votre première piste pour corriger.

> **Et l'IA générative dans tout ça ?** Colab propose des fonctions d'IA (bouton **« Generate »** ✨, ou l'assistant Gemini) qui écrivent du code à partir d'une simple description. Au **début**, mieux vaut **s'en passer** : c'est en écrivant vous-même que vous acquerrez les bases. Mais une fois que vous commencez à être **à l'aise en Python**, ces outils deviennent **très utiles et souvent très puissants**, à condition de toujours **relire, comprendre et vérifier** le code produit, comme le rappelle l'introduction de ce guide. Dans ces conditions, vous êtes fortement encouragés à vous en servir.

10. Se repérer : la table des matières



Pour naviguer facilement dans un notebook, Colab affiche une **table des matières** qui liste tous les titres de sections. Cliquez sur l'icône ☰ **(Table des matières)** tout en haut de la **barre latérale gauche** : un panneau s'ouvre avec la liste des sections. Un clic sur un titre vous y amène directement.